

Python Task 25-06-26

1. **Integer:** An Integer (int) in python is a data type which stores whole numbers including positive numbers negative numbers and zeros without decimals.

Features: 1. Immutability

2. No slicing allowed
3. Length cannot be found using len()
4. Stores whole numbers
5. Supports arithmetic operations

Ex: a=16

```
print(a)
print(type(a))
print(id(a))
```

```
#Immutable
a=20
print(id(a))
```

```
#stores whole numbers
b=-2
c=0
print(b)
print(c)
```

```
#Does not support indexing
print(a[0])
```

```
#Does not support length function
print(len(a))
```

```
#Arithmetic operations
x=10
y=5
print(x+y)
print(x-y)
print(x*y)
print(x/y)
```

o/p: 16

```
<class 'int'>
140734403057896
140734403058216 #20
-2
0
Error
Error
15
```

10
50
2.0

2.String:A string in python is sequence of characters used to store text data and enclosed between single(' ') ,double(" ") or triple quotes('' '').

Features:1.Immutable

- 2.Supports indexing and slicing
- 3.String length can be found using len().
- 4.Ordered
5. Iterable
- 6.Allows duplicate characters

Ex:

```
a="sivani"
```

```
print(a)
```

```
print(type(a))
```

```
#Immutable
```

```
print(a[2])="m"
```

```
#ordered
```

```
print(a)
```

```
#Indexing
```

```
print(a(2))
```

```
#Slicing
```

```
print(a[3:])
```

```
#iterable
```

```
For i in a:
```

```
    print(i)
```

```
#Duplication
```

```
word="class"
```

```
print(word)
```

```
#length
```

```
print(len(word))
```

o/p:sivani

```
<class 'str'>
```

```
Error
```

```
sivani
```

```
v
```

```
ani
```

```
s
```

```
i
v
a
n
i
class
5
```

3.List:A list in python is a collection of different data type elements enclosed between [].

Features:1.Allows heterogenous objects

2.Mutable

3.Ordered

4.Supports indexing and slicing

5.Iterable

6.Allows duplicate values

Ex: names = ["Sivani", "Suchitra", "Akshara",4,7.5,4,True]

```
print(names)
```

```
print(type(names))
```

```
#mutable
```

```
names[1] = "Anjali"
```

```
#Ordered ,Heterogenous and Duplicate values
```

```
print(names)
```

```
#Indexing
```

```
print(names[4])
```

```
#Slicing
```

```
print(names[1:3])
```

```
#Iterable
```

```
for i in names:
```

```
    print(i)
```

o/p:['Sivani', 'Suchitra', 'Akshara',4,7.5,4,True]

```
<class 'list'>
```

```
['Sivani', 'Anjali', 'Akshara',4,7.5,4,True]
```

```
7.5
```

```
['Anjali', 'Akshara']
```

```
Sivani
```

```
Anjali
```

```
4
```

```
7.5
```

```
4
```

True

4.Tuple:A tuple in python is a collection of elements enclosed between ().

Features:1.Immutable

2.Allows heterogenous values

3.Ordered

4.Supports indexing and slicing

5.Iterable

6.Allows duplicate values

Ex: n=(10,20,30,40,50,20)

```
print(n)
```

```
print(type(n))
```

```
Print(n[3])
```

```
print(n[2:5])
```

```
For i in n:
```

```
    print(i)
```

o/p: (10,20,30,40,50,20)

```
<class 'tuple'>
```

```
40
```

```
(30,40,50)
```

```
10
```

```
20
```

```
30
```

```
40
```

```
50
```

```
20
```

5.Set: A set in python is an unordered collection of unique elements enclosed by { }.

Features:1.Mutable

2. duplicate values are not allowed

3.unordered

4.Does not supports indexing and slicing

5.Iterable

Ex: numbers = {10, 20, 30, 40, 20, 50}

```
print(numbers)
```

```
#mutable
```

```
numbers.add(60)
```

```
#Duplicate and unordered
```

```
print(numbers)
```

```
Iterable
```

```
for i in numbers:
```

```
    print(i)
```

```
#Indexing
print(numbers[0])
```

```
#Slicing
print(numbers[1:3])
```

```
o/p:{10,20,30,40,50}
    {10,20,30,40,50,60}
    {10,20,30,40,50,60}
    10
    20
    30
    40
    50
    60
    Error
    Error
```

6.Dictionary:A dictionary in python stores key value pairs enclosed between { }.

Features:1.Mutable

2.Keys must be unique

3.Duplicate values are not allowed

4.Data can be accessed using keys

5.Ordered

6.Iterable

```
Ex:student = { "id": 101, "name": "Sivani", "course": "Python", "marks": 55,"grade": "A"}
print(student)
print(type(student))
```

```
#Mutable
student["marks"] = 78
print(student)
```

```
#keys must be unique
d={"id":101,"id":102}
print(d)
```

```
#Duplicate values are allowed
e={"math":90,"science":90}
print(e)
```

```
#ordered
print(student)
```

```
#Iterable
For key in student:
    print(key,":",student[key])
```

```
o/p:{'id': 101, 'name': 'Sivani', 'course': 'Python', 'marks': 55, 'grade': 'A' }
<class 'dict'>
{'id': 101, 'name': 'Sivani', 'course': 'Python', 'marks': 78, 'grade': 'A' }
{'id': 102}
{'math': 90, 'science': 90}
{'id': 101, 'name': 'Sivani', 'course': 'Python', 'marks': 78, 'grade': 'A'}
id : 101
name : Sivani
course : Python
marks : 78
grade : A
```

7.Mutable: The mutable data types in python are the data types that can be modified after they are created.They are:

- 1.List
- 2.Set
- 3.Dictionary

Immutable: Immutable data types in python are the data types that cannot be modified after they are created.They are:

- 1.Integer
2. String
3. Tuple

8.The data types that allow duplicate values are :

- 1.String
- 2.List
- 3.Tuple
- 4.Dictionary(only values can be duplicate but keys must be unique).

9.Insertion order: The following data types maintain insertion order in python.

- 1.String
- 2.List
- 3.Tuple
- 4.Dictionary

10.The data types in python that support indexing and slicing are :

- 1.String
- 2.List
- 3.Tuple.

11.The type function in python is used to find the data type of a variable or value in python.

Ex:

```
#Integer
a=4
```

```
#String
b="Sivani"
#List
c=[1,2,3]
#Tuple
d=(4,5,6)
#Set
e={65,45,62}
#Dictionary
f={"id": 101,"name": "Sivani"}
#Float
g=7.8
#Boolean
h=True
#Complex
i=3+6j
```

```
print(type(a))
print(type(b))
print(type(c))
print(type(d))
print(type(e))
print(type(f))
print(type(g))
print(type(h))
print(type(i))
```

```
o/p:<class 'int'>
      <class 'str'>
      <class 'list'>
      <class 'tuple'>
      <class 'set'>
      <class 'dict'>
      <class 'Float'>
      <class 'Bool'>
      <class 'complex'>
```

12.

Feature	int	str	list	tuple	set	dict
Definition	Stores whole numbers	Stores a sequence of characters	Stores a collection of elements	Stores a collection of elements	Stores unique elements	Stores data as key-value pairs
Mutable	No	No	Yes	No	Yes	Yes
Ordered	No	Yes	Yes	Yes	No	Yes
Allows Duplicate Values	Not Applicable	Yes	Yes	Yes	No	Only values can be duplicate. Keys must be unique
Supports Indexing	No	Yes	Yes	Yes	No	No
Supports Slicing	No	Yes	Yes	Yes	No	No
Iterable	No	Yes	Yes	Yes	Yes	Yes
Supports len()	No	Yes	Yes	Yes	Yes	Yes
Allows Heterogeneous Values	No	No	Yes	Yes	Yes	Yes
Example	10	"Sivani"	[1,2,3,4]	(1,2,3,4)	{1,2,3,4}	{id:"101"}